Research Plan

Constraint-Interaction Metric for Symbolic Regression

TIH IITG Summer Internship Project — July 6–28, 2026

Biswajyoti Nath & **Subinoy Nath**
Department of Computer Science & Engineering
Assam Science and Technology University

Abstract

We propose a **constraint-interaction metric** $M(i, j)$ for symbolic regression (SR) that quantifies whether pairs of search constraints act synergistically ($M > 1$), redundantly ($M < 1$), or independently ($M \approx 1$). The metric is defined as $M(i, j) = \rho(C_i \wedge C_j)/[\rho(C_i)\rho(C_j)]$, where $\rho(C)$ is the fraction of grammatically valid expressions satisfying constraint C. We will build a prototype system integrating a grammar-based expression generator, Monte Carlo density estimation, and PySR-based experiments on Feynman physics benchmarks. Deliverables include an open-source code repository, a 55–65 page implementation handbook, and empirical evidence that M predicts performance gains (target: Pearson $r > 0.6$, $p < 0.05$). Timeline: 23 working days (July 6–28, 2026).

Contents

1 Goals and Scope

1.1 Primary Goals

G1 Formally define $\rho(C)$ and the interaction coefficient $M(i, j)$.

G2 Implement efficient Monte Carlo algorithms to estimate these probabilities.

G3 Embed the framework in a symbolic regression workflow using PySR.

G4 Empirically validate that M predicts how constraint combinations affect search performance.

1.2 Scope

Given the 23-day timeline, we target a **focused prototype** producing convincing initial results suitable for a workshop or short conference submission. Specifically:

- A working system: data generators, metric calculator, experiment scripts.
- A moderate experiment set: 3–4 datasets, 4–5 constraint types, key pairwise combinations.
- Target: demonstrate $M(i, j)$ correlates with performance gains (Pearson $r > 0.6$, $p < 0.05$ after FDR correction) across representative SR problems.

1.3 Deliverables

Deliverable	Format
Implementation handbook	`Handbook.pdf` (55–65 pages)
Code repository	GitHub (Python modules, notebooks)
Sample datasets	`data/` folder (CSV)
Figures and results	`figures/`, `results/` (PNG/CSV)
README & CI	`README.md`, `.github/workflows/ci.yml`

Table 1: Project deliverables.

2 Background and Related Work

Symbolic regression (SR) discovers interpretable mathematical expressions fitting data. It is NP-hard due to the combinatorial search space. Researchers use *constraints*—grammar rules, type systems, operator whitelists, dimensional analysis—to incorporate domain knowledge and reduce search effort.

2.1 Prior Work on Constraints in SR

- **Grammar-based SR:** Attribute grammars enforcing dimensional consistency recovered ∼60% more benchmark equations (Brence et al., 2023).
- **Semantic backpropagation:** Reissmann et al. (2025) use semantic constraints during evolution in GEP.
- **Graph-based SR:** Xiang et al. (2025) encode constraints into neural-guided MCTS.
- **PySR:** State-of-the-art open-source SR using multi-population evolutionary search (Cranmer, 2023).

2.2 Research Gap

Novelty Statement

All prior work studies individual constraint effectiveness. **No existing work models pairwise statistical interaction between constraints as a predictive signal for search performance.** Our metric $M(i, j)$ fills this gap. While analogous to "lift" in association rule mining, the application to structured expression grammars—where co-occurrence is measured over a *generative* distribution rather than an observed corpus—is novel in the SR literature.

3 Proposed Metric

3.1 Definitions

Let $\mathcal{E}$ be the space of expressions generated by grammar G. For a constraint $C : \mathcal{E} \to \{0, 1\}$, define:

$$\rho(C) \;=\; \Pr_{E \sim G}[C(E) = 1] \tag{1}$$

The **interaction coefficient** for constraints C_i, C_j is:

$$\boxed{M(i, j) \;=\; \frac{\rho(C_i \wedge C_j)}{\rho(C_i)\,\rho(C_j)}} \tag{2}$$

3.2 Interpretation

- $M(i, j) > 1$: **Synergy** — constraints co-occur more than chance; combining them may yield super-additive gains.
- $M(i, j) < 1$: **Redundancy** — one constraint subsumes the other; adding the second provides diminishing returns.
- $M(i, j) = 1$: **Independence** — constraints act on orthogonal dimensions.

3.3 Statistical Significance of Interaction

Rather than using arbitrary thresholds ($M > 1.1$, $M < 0.9$), we determine significance via a bootstrap confidence interval. Under the null hypothesis of independence, $M = 1$. We:

1. Bootstrap-resample the expression set $B = 1000$ times.
2. Compute $\hat{M}^{(b)}$ for each resample.
3. Construct a 95% CI for M. If $1 \notin$ CI, reject independence.

4 Data Generation

4.1 Grammar-Based Expression Generator

We use a context-free grammar in BNF:

```
<expr> ::= <expr> <binop> <expr>
         | <unop>(<expr>)
         | <variable> | <constant>
<binop> ::= + | - | *
```

```
<unop>   ::= sin | cos
<variable> ::= x | y
<constant> ::= 1 | 2 | ... | 9
```

A recursive generator applies grammar rules up to `max_depth`. Division is excluded from the default grammar to avoid domain errors; it can be added as a configurable extension.

4.2 Sampling Strategy

Parameter	Value	Rationale
Grammar depth limit	8	Moderate complexity
Operators	$+, -, \times, \sin, \cos$	Core arithmetic + trig
Variables	x, y	Two-input functions
Constants	$\pm 1 \ldots 9$	Discrete values
Sample size N	100,000	$\rho \sim 0.01 \Rightarrow \sim$1000 hits, SE $\sim$3%
Stratification	Equal samples per depth 2,4,6,8	Avoid depth bias

Table 2: Monte Carlo sampling parameters.

Each expression string is converted to a SymPy object via `sympify()`, with invalid expressions (syntax errors, domain violations) discarded and replaced.

5 Constraint Definitions

We implement five constraints $C(E)$, designed to be **structurally distinct**:

C1 Structural Template (C_{struct}): The expression must conform to a structural pattern—e.g., no nested trigonometric calls, at most 2 consecutive binary operations. This is checked by traversing the SymPy AST.

C2 Maximum Depth (C_{depth}): Parse-tree depth $\leq$ threshold (default: 6). Recursive depth computation on `sym_expr`.

C3 Operator Whitelist (C_{op}): Only operators from a prescribed set (e.g., $\{+, -, \times\}$) may appear. Every AST node is verified against the whitelist.

C4 Positivity (C_{pos}): The expression evaluates to ≥ 0 for all inputs in $[-10, 10]^2$. We test on **200 random points** (not 10) and report the empirical false-positive rate: $P(\text{pass} \mid \text{violates}) \leq (0.95)^{200} < 10^{-4}$ if 5% of the domain violates.

C5 Dimensional Consistency (C_{dim}, optional): Each variable carries a physical dimension; we propagate units through operations and reject inconsistencies (e.g., length+time). Skipped if time is short.

Design Decision: C1 vs. C3

C_{struct} and C_{op} are intentionally distinct: the former constrains *tree shape* (nesting patterns), the latter constrains *node labels* (which operators appear). Their M value is therefore non-trivial.

6 Monte Carlo Estimation

Algorithm 1: Monte Carlo Constraint Density Estimation

Input: Grammar G, constraints $\{C_1, \ldots, C_k\}$, sample size N
Output: Densities $\hat{\rho}(C_i)$, joint densities $\hat{\rho}(C_i \wedge C_j)$, matrix M
Initialize counters $n_i \leftarrow 0$, $n_{ij} \leftarrow 0$;
for $t = 1$ **to** N **do**
  $E \leftarrow$ `generate_expr`$(G, \texttt{max\_depth} = 8)$;
  $s \leftarrow$ `sympify`(E);
  if s *is invalid* **then**
    continue (replace sample);
  end
  for $i = 1$ **to** k **do**
    if $C_i(s)$ **then**
      $n_i \leftarrow n_i + 1$;
    end
    for $j = i + 1$ **to** k **do**
      if $C_i(s) \wedge C_j(s)$ **then**
        $n_{ij} \leftarrow n_{ij} + 1$;
      end
    end
  end
end
$\hat{\rho}(C_i) \leftarrow n_i / N$;
$\hat{\rho}(C_i \wedge C_j) \leftarrow n_{ij} / N$;
$M(i, j) \leftarrow \hat{\rho}(C_i \wedge C_j) / [\hat{\rho}(C_i) \cdot \hat{\rho}(C_j)]$;
return $\hat{\rho}, M$

Parallelism: Sampling is embarrassingly parallel. We use Python's `multiprocessing` or `joblib` to distribute across cores.

Convergence check: Run at $N = 1\text{k}, 10\text{k}, 100\text{k}$ and verify $|\hat{\rho}_{100\text{k}} - \hat{\rho}_{10\text{k}}| < 0.01$.

7 Software Architecture

7.1 Module Structure

Module	Responsibility
`expr_generator.py`	Grammar-based recursive expression generation
`constraints.py`	Constraint implementations (C1–C5) with unit tests
`metric.py`	Monte Carlo sampling, ρ estimation, M computation
`experiments.py`	PySR integration, experiment orchestration, logging
`analysis.py`	Statistical analysis, correlation, regression, plotting
`config.yaml`	All parameters (grammar, constraints, PySR, seeds)
`main.py`	CLI entry point: `--phase={metric,experiment,analysis}`

Table 3: Module structure.

7.2 Configuration Schema

Listing 1: Minimal `config.yaml` schema.

```yaml
grammar:
  max_depth: 8
  operators: ["+", "-", "*", "sin", "cos"]
  variables: ["x", "y"]
monte_carlo:
  N: 100000
  stratify_by_depth: true
pysr:
  population_size: 40
  niterations: 100
  timeout_seconds: 300
datasets: ["feynman_1", "feynman_2", "polynomial"]
seeds: [42, 123, 456, 789, 1024]
```

7.3 Testing Strategy

- **Unit tests:** Known pass/fail expressions for each constraint.
- **Convergence test:** Assert ρ stabilizes within 5% between $N = 10$k and $N = 100$k.
- **Integration test:** End-to-end pipeline on a toy 3-operator grammar.

7.4 Dependency Management

All dependencies pinned in `requirements.txt`: `pysr>=0.19`, `sympy==1.12`, `numpy>=1.24`, `scipy>=1.11`, `seaborn>=0.13`, `pandas>=2.0`.

8 Experimental Design

8.1 Datasets

D1 Feynman-1: Simple physics law (e.g., $E = mc^2$, kinetic energy).

D2 Feynman-2: Multi-variable with trigonometry (e.g., projectile motion).

D3 Polynomial: Synthetic 3-term polynomial.

D4 SRSD: Dataset with dummy variable (Matsubara et al., 2024).

8.2 Constraint Scenarios

For each dataset, PySR is run under:

- **Baseline:** No additional constraints.
- **Singles:** Each of C1–C4 individually.
- **Key pairs:** Pairs with extreme predicted M values (highest synergy, highest redundancy).
- **All constraints:** C1–C4 combined.

Each scenario uses **5 repeats** (fixed seeds) for statistical reliability.

8.3 Constraint Enforcement in PySR

Constraints are enforced **during search** via PySR's `constraints` and `nested_constraints` parameters (for structural/operator constraints) and custom loss penalties (for positivity). This ensures the metric captures genuine search-effort reduction, not post-hoc filtering.

8.4 Success Criterion

Recovery is defined as: NED < 0.1 **or** relative MSE $< 10^{-6}$ on 20% held-out data.

8.5 Metrics Recorded

- Wall-clock runtime (seconds)
- Number of candidate evaluations
- Recovery rate (binary)
- Solution complexity (AST node count)

9 Statistical Analysis

9.1 Speedup Definition

For constraint pair (i, j), the **speedup** compares combined performance against the best single:

$$\boxed{S(i,j) = \frac{\min(\text{evals}_{C_i},\ \text{evals}_{C_j})}{\text{evals}_{C_i \wedge C_j}}} \tag{3}$$

$S > 1$ means the combination outperforms either constraint alone.

9.2 Correlation Analysis

- Pearson and Spearman correlation between $M(i,j)$ and $S(i,j)$.
- Linear regression: $S(i,j) = a + b \cdot M(i,j) + \varepsilon$. Significant positive b ($p < 0.05$) supports predictive value.
- **Multiple comparisons:** Benjamini–Hochberg FDR correction across all pairwise tests.

9.3 Confidence Intervals

95% bootstrap CIs for all mean metrics. Error bars on all plots.

9.4 Power Considerations

With $\binom{4}{2} \times 4 = 24$ (pair, dataset) data points, a power analysis (`statsmodels.stats.power`) confirms detectability of $r \geq 0.6$ at $\alpha = 0.05$ with power > 0.80.

10 Visualization Plan

1. **Interaction Heatmap:** $M(i,j)$ matrix with `seaborn.heatmap`, annotated cells, cool-warm colormap.
2. **Scatter Plot:** $M(i,j)$ vs. $S(i,j)$ with regression line and r/p annotation.
3. **Box Plots:** Runtime/evaluations per constraint scenario per dataset.
4. **Convergence Plot:** $\hat{\rho}(C)$ vs. sample size N.

All generated with `matplotlib/seaborn`; publication-quality (300 DPI, vector where possible).

11 Potential Pitfalls and Mitigations

Risk	Impact	Mitigation
Sampling noise for rare ρ	Inaccurate M	Increase N; report bootstrap CIs
Generator $\neq$ search distribution	M may not transfer	Compare with PySR gen-0 population
SymPy edge cases	Constraint bugs	Unit tests on known expressions
PySR stochasticity	Wide CIs	5 repeats; fix seeds
PySR installation failure	Blocked experiments	Fallback to `gplearn`
Timeline slippage	Incomplete deliverables	2 buffer days; cut C_{dim} first

Table 4: Risk register.

12 Timeline

Day	Date	Tasks	Hours
1	Jul 6	Setup env; install Julia+PySR; review literature; begin generator	6
2	Jul 7	Finish generator; SymPy integration; depth-limit tests	8
3	Jul 8	Monte Carlo sampling; validity checks; benchmark 1k samples	8
4	Jul 9	Implement C1 (structural), C2 (depth), C3 (operator)	7
5	Jul 10	Implement C4 (positivity, 200 pts); optional C5 (dimensional)	8
6	Jul 11	Full Monte Carlo run (N=100k); convergence check	8
7	Jul 12	Compute M matrix; heatmap visualization; identify patterns	6
8	Jul 13	Document theory (ρ, M, bootstrap CI); write handbook §1–4	7
9	Jul 14	Software architecture; module APIs; pseudocode in handbook	6
10	Jul 15	PySR setup; prepare benchmark datasets; data loading scripts	6
11	Jul 16	Integrate constraints with PySR API; plan enforcement strategy	7

Day	Date	Tasks	Hours
12	Jul 17	Baseline SR experiments (no constraints); debug; store results	8
13	Jul 18	Single-constraint experiments (5 repeats each); log metrics	8
14	Jul 19	Pair and all-constraint experiments; focus on extreme M pairs	8
15	Jul 20	Statistical analysis: means, CIs, correlations, FDR correction	6
16	Jul 21	Ablation studies; begin drafting Results chapter	6
17	Jul 22	Create all figures (heatmap, scatter, boxplots)	7
18	Jul 23	Write Experiments & Results chapters	8
19	Jul 24	Write Implementation Details chapter	8
20	Jul 25	Write Data Generation & Constraints chapters	7
21	Jul 26	*Buffer day:* catch-up on any delayed tasks	6
22	Jul 27	Finalize writing; proofread; compile handbook PDF	6
23	Jul 28	*Buffer + submit:* final polish, push repo, CI, README	4

Table 5: Daily timeline (July 6–28, 2026). Days 21 and 23 are buffer days.

13 Handbook Outline

Target: **55–65 pages**, ∼25,000 words.

1. Cover Page (0.5 p.)
2. Abstract & Executive Summary (1 p.)
3. Table of Contents (1 p.)
4. Ch. 1: Introduction (2 p.) — SR background, motivation, contributions.
5. Ch. 2: Related Work (3 p.) — SR techniques, constraints, benchmarks.
6. Ch. 3: Problem Definition (2 p.) — Formal $M(i, j)$, research questions.
7. Ch. 4: Proposed Metric (4 p.) — Derivation, interpretation, bootstrap CI.
8. Ch. 5: Software Architecture (5 p.) — Modules, data flow, config schema.
9. Ch. 6: Data & Expression Generation (4 p.) — Grammar, sampling, stratification.
10. Ch. 7: Constraint Definitions (4 p.) — Formal + implementation.
11. Ch. 8: Monte Carlo Estimation (3 p.) — Algorithm, convergence, parallelism.
12. Ch. 9: PySR Integration (4 p.) — API usage, constraint enforcement.
13. Ch. 10: Experimental Protocol (4 p.) — Datasets, experiment matrix, metrics.
14. Ch. 11: Statistical Methods (3 p.) — Correlation, regression, FDR, power.
15. Ch. 12: Results (5 p.) — Heatmap, scatter plots, tables.
16. Ch. 13: Discussion (4 p.) — Interpretation, limitations.

17. Ch. 14: Conclusion & Future Work (2 p.)
18. Appendices (4 p.) — Code listings, extended tables.

References

[1] M. Reissmann et al., "Constraining Genetic Symbolic Regression via Semantic Backpropagation," *Genetic Programming and Evolvable Machines*, 2025.

[2] Y. Xiang et al., "Graph-based Symbolic Regression with Invariance and Constraint Encoding," *NeurIPS*, 2025.

[3] M. Cranmer, "Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl," *arXiv:2305.01582*, 2023.

[4] J. Brence et al., "Dimensionally-consistent equation discovery through probabilistic attribute grammars," *Information Sciences*, vol. 632, 2023.

[5] Y. Matsubara et al., "Rethinking Symbolic Regression Datasets and Benchmarks for Scientific Discovery," *J. Data-centric Machine Learning Research*, 2024.

[6] J. Martin and M. Burnett, "Shape-Constrained Symbolic Regression—Improving Extrapolation," in *GECCO*, 2014.

[7] PySR Documentation and GitHub, 2023. https://ai.damtp.cam.ac.uk/pysr/